МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра МОЭВМ

ОТЧЕТ

по лабораторной работе №2

по дисциплине «Построение и Анализ Алгоритмов»

Тема: Задача коммивояжера, построение МОД

Студент гр. 4342 Ларионов В.А.

Преподаватель Уткина О.Н.

Санкт-Петербург

2026

Задание.

Вариант 7

Точный метод: МВиГ – построить минимальное остовное дерево (далее – МОД) графа с дополнительным условием: задана максимальная степень (от 2 до |V|−1), допустимая для всех вершин в требуемом МОД (если эта максимальная степень равна 2, то получится обычная незамкнутая ЗК, а если |V|−1, то получится обычная задача построения МОД).

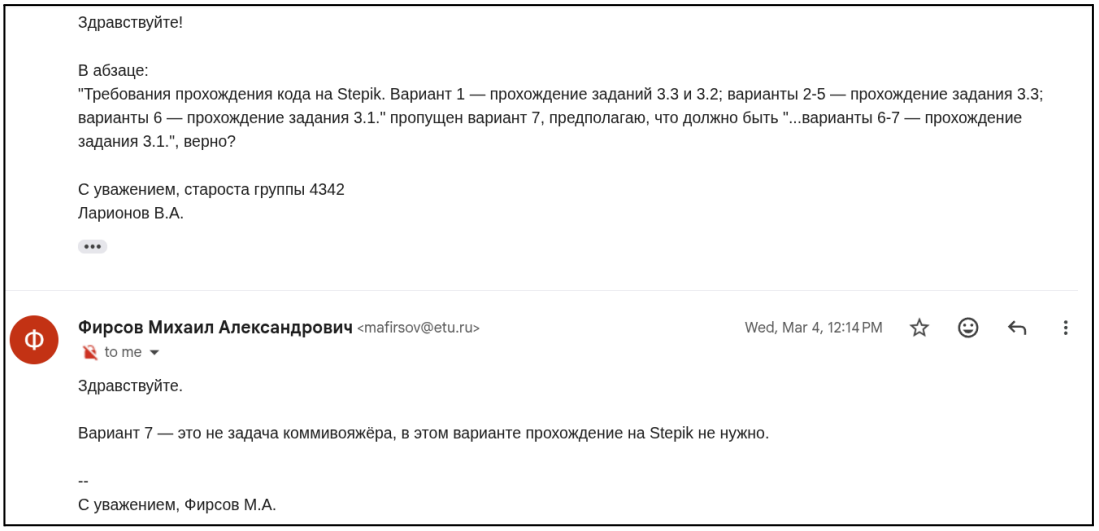
Использовать для отсечения ветвей сумму веса частичного решения и нижней оценки веса недостающих ребер решения на основе МОД (без ограничения степени).

Дано: симметричная матрица весов графа, все веса неотрицательны; максимальная степень вершин МОД (от 2 до |V|−1).

Найти: МОД графа с ограничением степени и его стоимость.

Приближенный алгоритм: аналог алгоритма Краскала, но ребро не добавляется, если его добавление приводит к нарушению ограничения по степени. Приближенный алгоритм применяется только к полным графам.

В связи с тем, что в варианте 7 задача коммивояжера является частным случаем построения МОД, прохождение задания на Stepik не требуется, что подтверждается письмом преподавателя на Скриншоте 1.



Здравствуйте!

В абзаце:
"Требования прохождения кода на Stepik. Вариант 1 — прохождение заданий 3.3 и 3.2; варианты 2-5 — прохождение задания 3.3; варианты 6 — прохождение задания 3.1." пропущен вариант 7, предполагаю, что должно быть "...варианты 6-7 — прохождение задания 3.1.", верно?

С уважением, староста группы 4342
Ларионов В.А.

•••

Фирсов Михаил Александрович <mafirsov@etu.ru> Wed, Mar 4, 12:14 PM
to me

Здравствуйте.

Вариант 7 — это не задача коммивояжёра, в этом варианте прохождение на Stepik не нужно.

--
С уважением, Фирсов М.А.

Скриншот 1 - Ответ преподавателя про 7 вариант

Описание алгоритма.

Задача состоит в поиске МОД в графе с неотрицательными весами при дополнительном ограничении: степень любой вершины в результирующем дереве не должна превышать заданного значения MAX_VERTEX_DEG (от 2 до |V|−1). При MAX_VERTEX_DEG = 2 задача сводится к задаче коммивояжера, при MAX_VERTEX_DEG = |V| - 1 -- к классической задаче построения МОД.

Для решения применяется точный метод ветвей и границ (МВиГ). Его основная идея -- последовательное перебирание рёбер, отсортированных по возрастанию веса, с формированием частичного решения. В каждой вершине дерева перебора выполняются отсечения на основе нижней оценки, вычисляемой как вес уже включенных рёбер плюс вес МОД, построенного на оставшихся ребрах без учета ограничения степени. Нижняя оценка получается с помощью классического алгоритма Краскала, работающего на копии системы непересекающихся множеств (DSU) текущего частичного решения.

Дополнительно реализован приближенный алгоритм, представляющий собой модифицированный алгоритм Краскала: ребро добавляется в дерево, только если его включение не приводит к превышению MAX_VERTEX_DEG у инцидентных вершин. Этот алгоритм дает допустимое решение (возможно, не оптимальное) и используется для инициализации верхней границы (bestWeight) в точном методе. Согласно заданию, приближенный алгоритм применяется только к полным графам; на неполных графах он может не построить остов, вернув бесконечный вес.

Оптимизации алгоритма.

1. Сортировка рёбер.

Все рёбра графа однократно сортируются по весу. Это позволяет в точном методе рассматривать их в порядке возрастания и при вычислении нижней оценки передавать подмассив оставшихся рёбер без повторной сортировки.

2. Эвристики DSU.

В структуре DSU используются сжатие путей и объединение по рангу, что обеспечивает практически константное время выполнения операций find и unite.

3. Отсечение по нижней границе.

Перед попыткой включить очередное ребро вычисляется нижняя граница веса достроенного дерева. Если текущий_вес + нижняя_оценка ≥ bestWeight, ветвь отсекается.

4. Использование приближенного решения.

Вес дерева, полученного приближенным алгоритмом, служит начальным значением bestWeight. Это позволяет отсечь большое количество заведомо неоптимальных ветвей уже на ранних этапах перебора.

5. Раннее обнаружение циклов.

Благодаря DSU циклы выявляются за O(α(V)), что быстрее явного обхода графа. Отсечение при нарушении связности. Если функция kruskalLowerBound возвращает inf, это означает, что из оставшихся ребер невозможно построить остовное дерево, и ветвь немедленно отсекается.

Сложность алгоритма.

Приближенный алгоритм: сортировка рёбер -- $O(E\,log\,E)$, где $E = \frac{V(V-1)}{2}$ для полного графа.

Точный метод: в худшем случае перебирает все подмножества ребер, что дает экспоненциальную сложность $O(2^E)$. Однако благодаря сильным отсечениям (нижняя оценка на основе МОД, начальное приближение) на практике алгоритм часто работает значительно быстрее.

Описание функций и структур данных.

struct edge - хранит пару вершин (std::pair<int,int>) и вес ребра. Определён оператор == для поиска в векторе.

struct DSU - система непересекающихся множеств с методами find (сжатие путей) и unite (объединение по рангу). Используется для отслеживания компонент связности и предотвращения циклов.

getMatrixFromStream() - считывает матрицу весов из стандартного ввода.

isMatrixSymmetric() - проверяет матрицу на симметричность.

getEdgesStack() - формирует список всех рёбер с ненулевым весом из матрицы смежности (каждое ребро включается один раз).

sortEdgesStack() - сортирует список рёбер по возрастанию веса.

kruskalLowerBound() - вычисляет вес МОД (или его нижнюю оценку) на заданном наборе рёбер с учётом начального разбиения на компоненты (dsu). При digitLimit = true дополнительно контролирует степень вершин (используется только для получения начального приближения). Может возвращать inf, если остов построить невозможно.

findMSTRecursive() - рекурсивная функция точного поиска. Реализует метод ветвей и границ: проверяет полноту решения, вычисляет нижнюю границу, пытается включить текущее ребро (если степени позволяют и нет цикла) и рекурсивно вызывает себя для оставшихся рёбер. Обновляет глобально лучшие вес и список ребер.

printResult() - форматирует и выводит в консоль точное и (при расхождении) приближенное решения.

edgesToDot() - генерирует файл в формате Graphviz DOT для визуализации графа с выделением рёбер найденного МОД.

findMSTBranchAndBound() - основная управляющая функция: загружает данные, запускает приближенный алгоритм, затем точный метод, выводит результаты.

main() - проверяет макрос MAX_VERTEX_DEG и вызывает главную функцию.

Тестирование.

Тестирование приведено в таблице 1 в Приложении Б.

ПРИЛОЖЕНИЕ А

ИСХОДНЫЙ КОД ПРОГРАММЫ

Файл main.cpp

```
#include <iostream>
#include <vector>
#include <fstream>
#include <algorithm>
#include <string>
#include <sstream>
#include <limits>

#define GRAPH_FILENAME "./files/graph.dot"
#ifndef MAX_VERTEX_DEG
#define MAX_VERTEX_DEG 5
#endif
#ifndef SHOW_INFO
    #define SHOW_INFO 0
#endif
#if SHOW_INFO
    #define LOG_FILENAME "./files/log.txt"
    std::ofstream logFile(LOG_FILENAME);
    int logCounter = 0;
#endif


const std::string ALPHABET = \
"ABCDEFGHIJKLMNOPQRSTUVWXYZabcdefghijklmnopqrstuvwxyz";


/**
 * Структура ребра графа.
 * Содержит пару вершин (индексы) и вес.
 * Определён оператор == для поиска в векторе.
 */
struct edge {
    std::pair<int, int> vertices;
    int weight;
    bool operator==(const edge& counter) const {
        return vertices == counter.vertices && weight == counter.weight;
    }
};


using edgesStack = std::vector<edge>;
using matrix = std::vector<std::vector<int>>;


#if SHOW_INFO
void infoToLogFile(const std::string& info) {
    if (logFile) {
        logFile << logCounter++ << "\n";
        std::stringstream ss(info);
        std::string token;
        while (std::getline(ss, token, '\n')) {
            logFile << "\t" << token << std::endl;
        }
    }
}


/**
```

```
 * Преобразование матрицы весов в строку с буквенными обозначениями вершин.
 */
std::string matrixToString(const matrix& M) {
    std::string strMatrix;
    size_t squareMatrixSide = M.size();
    for (size_t i = 0; i < squareMatrixSide; ++i) {
        strMatrix += "\t";
        strMatrix += ALPHABET[i];
    }
    strMatrix += "\n\n";
    for (size_t i = 0; i < squareMatrixSide; ++i) {
        strMatrix += ALPHABET[i];
        strMatrix += "\t";
        for (size_t j = 0; j < squareMatrixSide; ++j) {
            strMatrix += std::to_string(M[i][j]);
            strMatrix += "\t";
        }
        strMatrix += "\n\n";
    }
    return strMatrix;
}


/**
 * Возвращает строку вида "A -- B" для ребра между вершинами u и v.
 */
std::string verticesToStringEdge(const int vertexU, const int vertexV) {
    std::stringstream ss;
    ss << ALPHABET[vertexU] << " -- " << ALPHABET[vertexV];
    return ss.str();
}


/**
 * Формирует многострочное текстовое представление списка рёбер.
 */
std::string edgesToString(const edgesStack& edges) {
    std::string strCurrentEdges;
    for (const edge& edge : edges) {
        int vertexU = edge.vertices.first;
        int vertexV = edge.vertices.second;
        strCurrentEdges += verticesToStringEdge(vertexU, vertexV);
        strCurrentEdges += "\n";
    }
    return strCurrentEdges;
}


/**
 * Возвращает строковое представление веса: число либо "inf".
 */
std::string bestWeightString(const int& bestWeight) {
    return bestWeight == std::numeric_limits<int>::max() ?
    "inf" : std::to_string(bestWeight);
}
#endif


/**
  * Система непересекающихся множеств (DSU) с эвристиками сжатия путей и
объединения по рангу.
 * Используется в алгоритме Краскала для проверки циклов и объединения компонент
связности.
```

```
 */
struct DSU {
    std::vector<int> sets, rank;
    DSU(int setsCount) : sets(setsCount), rank(setsCount, 0) {
        for (int i = 0; i < setsCount; ++i)
            sets[i] = i;
    }
    /**
     * Находит представителя множества для элемента indexInSet.
     * Применяется сжатие путей.
     */
    int find(int indexInSet) {
        if (indexInSet == sets[indexInSet]) {
            return indexInSet;
        }
        return sets[indexInSet] = find(sets[indexInSet]);
    }
    /**
     * Объединяет множества, содержащие setA и setB.
        * Возвращает true, если объединение выполнено (элементы были в разных
множествах),
     * иначе false.
     */
    bool unite(int setA, int setB) {
        setA = find(setA);
        setB = find(setB);
        if (setA == setB) {
            return false;
        }
        if (rank[setA] < rank[setB]) {
            std::swap(setA, setB);
        }
        sets[setB] = setA;
        if (rank[setA] == rank[setB]) {
            rank[setA]++;
        }
        return true;
    }
};


/**
 * Считывает матрицу весов из стандартного ввода.
 * Формат: первое число – количество вершин N,
 * затем N строк по N чисел (веса рёбер).
 */
matrix getMatrixFromStream() {
    int verticesCount;
    std::cin >> verticesCount;
    matrix M(verticesCount, std::vector<int>(verticesCount));
    for (int i = 0; i < verticesCount; ++i) {
        for (int j = 0; j < verticesCount; ++j) {
            std::cin >> M[i][j];
        }
    }
    return M;
}


/**
 * Проверяет, содержится ли заданное ребро в списке (сравнение по ==).
 */
bool isEdgeIn(const edgesStack& edges, const edge& edge) {
```

```
    return std::find(edges.begin(), edges.end(), edge) == edges.end();
}


/**
 * Генерирует файл в формате Graphviz DOT, отображающий исходный граф
 * и выделяющий рёбра МОД зеленым цветом.
 */
void edgesToDot(
    const edgesStack& edges,
    const edgesStack& MSTedges,
    const std::string& filename
) {
    std::ofstream dotFile(filename);
    if (!dotFile.is_open()) {
        return;
    }
    std::string dotInfoString("graph G {\n\tnode [shape=circle];\n");
    for (const auto& currentEdge : edges) {
        const int& vertexU = currentEdge.vertices.first;
        const int& vertexV = currentEdge.vertices.second;
        dotInfoString += "\t";
        dotInfoString += ALPHABET[vertexU];
        dotInfoString += " -- ";
        dotInfoString += ALPHABET[vertexV];
        dotInfoString += " [label=\"";
        dotInfoString += std::to_string(currentEdge.weight);
        if (isEdgeIn(MSTedges, currentEdge)) {
            dotInfoString += "\" color=\"red\" penwidth=\"1\"];\n";
        }
        else {
            dotInfoString += "\" color=\"green\" penwidth=\"2\"];\n";
        }
    }
    dotInfoString += "}\n";
    dotFile << dotInfoString;
}


/**
 * Проверяет симметричность матрицы.
 */
bool isMatrixSymmetric(const matrix& M) {
    for (size_t i = 0; i < M.size(); ++i) {
        for (size_t j = i + 1; j < M.size(); ++j) {
            if (M[i][j] != M[j][i]) {
                return false;
            }
        }
    }
    return true;
}


/**
 * Преобразует матрицу смежности в список рёбер.
 * Включаются только рёбра с ненулевым весом (i < j, чтобы избежать дублирования).
 */
edgesStack getEdgesStack(const matrix& M) {
    edgesStack edges;
    for (size_t i = 0; i < M.size(); ++i) {
        for (size_t j = i + 1; j < M.size(); ++j) {
            if (M[i][j]) {
```

```
                edges.push_back({{{(int)i}, {(int)j}}, M[i][j]});
            }
        }
    }
    return edges;
}


/**
 * Сортирует список рёбер по возрастанию веса.
 */
void sortEdgesStack(edgesStack& edges) {
    auto comporator = [](const edge& a, const edge& b) {
        return a.weight < b.weight;
    };
    std::sort(edges.begin(), edges.end(), comporator);
}


/**
 * Вычисляет вес минимального остовного дерева (МОД) на основе алгоритма Краскала.
 * Используется как для получения приближённого решения (с ограничением степени),
 * так и для вычисления нижней оценки в методе ветвей и границ (без ограничения
степени).
 *
 * Параметры:
 *   edges         -- отсортированный по весу список доступных рёбер.
 *     dsu                   -- текущее состояние компонент связности (копия,
модифицируется).
 *   verticesCount -- общее число вершин графа.
 *   digitLimit    -- если true, учитывается ограничение MAX_VERTEX_DEG на степень
вершин.
 *   mstEdges      -- если не nullptr, в него записываются рёбра построенного МОД.
 *
 * Возвращает вес построенного дерева либо std::numeric_limits<int>::max(),
 * если построить дерево невозможно (не хватает рёбер или нарушены ограничения).
 */
int kruskalLowerBound(
    edgesStack& edges,
    DSU dsu,
    int verticesCount,
    bool digitLimit = false,
    edgesStack* mstEdges = nullptr
) {
    int MSTweight = 0;
    int currentVertexNumber = 0;
    std::vector<int> rank(edges.size(), 0);
    for (int i = 0; i < verticesCount; ++i) {
        if (dsu.find(i) == i) {
            currentVertexNumber++;
        }
    }
    currentVertexNumber = verticesCount - currentVertexNumber;
    for (const auto& currentEdge : edges) {
        int vertexU = currentEdge.vertices.first;
        int vertexV = currentEdge.vertices.second;
        if (digitLimit) {
            if (std::max(rank[vertexU], rank[vertexV]) >= MAX_VERTEX_DEG) {
                continue;
            }
        }
        if (dsu.unite(vertexU, vertexV)) {
            if (digitLimit) {
```

```
                rank[vertexU]++; rank[vertexV]++;
            }
            MSTweight += currentEdge.weight;
            currentVertexNumber++;
            if (mstEdges != nullptr) {
                mstEdges->push_back(currentEdge);
            }
            if (currentVertexNumber == verticesCount - 1) {
                return MSTweight;
            }
        }
    }
    if (currentVertexNumber != verticesCount - 1) {
        return std::numeric_limits<int>::max();
    }
    return MSTweight;
}


/**
 * Рекурсивная функция поиска точного решения методом ветвей и границ.
 * Перебирает рёбра в порядке сортировки, формируя допустимые остовные деревья
 * с ограничением степени MAX_VERTEX_DEG.
 *
 * Параметры:
 *   edges         -- полный список рёбер (отсортирован).
 *   index         -- текущий индекс рассматриваемого ребра.
 *   degree        -- текущие степени вершин в частичном решении.
 *    dsu               -- состояние компонент связности для частичного решения
(копия).
 *   currentWeight -- суммарный вес рёбер в частичном решении.
 *   bestWeight     -- вес лучшего найденного на данный момент решения (изменяется
при улучшении).
 *   currentEdges  -- рёбра текущего частичного решения.
 *   bestEdges     -- рёбра лучшего решения (обновляется вместе с bestWeight).
 *   verticesCount -- общее число вершин.
 */
void findMSTRecursive(
    edgesStack& edges,
    int index,
    std::vector<int>& degree,
    DSU dsu,
    int currentWeight,
    int& bestWeight,
    edgesStack& currentEdges,
    edgesStack& bestEdges,
    int verticesCount
) {
    #if SHOW_INFO
    std::string info("Вызвана рекурсивная функция findMSTRecursive\n");
    info += "Вес лучшего решения: ";
    info += bestWeightString(bestWeight);
    info += "\nВес текущего решения: ";
    info += std::to_string(currentWeight);
    info += "\nРебра в текущем решении:\n";
    info += edgesToString(currentEdges);
    infoToLogFile(info);
    #endif
    // Если собрано V-1 рёбер, проверяем на улучшение рекорда
    if (currentEdges.size() == verticesCount - 1) {
        #if SHOW_INFO
        info.clear();
        info += "Текущее решение содержит ";
```

```
        info += std::to_string(verticesCount - 1);
        info += " рёбер, оно окончательно для графа из ";
        info += std::to_string(verticesCount);
        info += " вершин.\nПроверим лучше ли оно предыдущего:";
        infoToLogFile(info);
        #endif
        if (currentWeight < bestWeight) {
            #if SHOW_INFO
            info.clear();
            info += "Да, currentWeight = ";
            info += std::to_string(currentWeight);
            info += ", а bestWeight = ";
            info += bestWeightString(bestWeight);
            info += ". Заменяем лучшее решение текущим.";
            infoToLogFile(info);
            #endif
            bestWeight = currentWeight;
            bestEdges = currentEdges;
        }
        #if SHOW_INFO
        else {
            info.clear();
            info += "Нет, currentWeight = ";
            info += std::to_string(currentWeight);
            info += ", а bestWeight = ";
            info += std::to_string(bestWeight);
            info += ". Оставляем все как есть.";
            infoToLogFile(info);
        }
        #endif
        return;
    }
    if (index >= edges.size()) {
        #if SHOW_INFO
        infoToLogFile("Index out of range, отсекаю ветвь рекурсии");
        #endif
        return; // рёбра кончились, дерево не достроено
    }
    edgesStack remaining(edges.begin() + index, edges.end());
    int bound = kruskalLowerBound(remaining, dsu, verticesCount);
    if (bound == std::numeric_limits<int>::max()) {
        #if SHOW_INFO
        info.clear();
        info += "Оценка МОД на оставшихся вершинах методом Крускала показала,";
        info += "\nчто построить МОД с текущими вершинами невозможно,";
        info += "\nотсекаю эту ветвь рекурсии.";
        infoToLogFile(info);
        #endif
        return; // невозможно достроить до остова
    }
    if (currentWeight + bound >= bestWeight) {
        #if SHOW_INFO
        info.clear();
        info += "currentWeight = ";
        info += std::to_string(currentWeight);
        info += "\tbound = ";
        info += std::to_string(bound);
        info += "\tbestWeight = ";
        info += std::to_string(bestWeight);
        info += "\n";
        info += std::to_string(currentWeight);
        info += " + ";
        info += std::to_string(bound);
```

```
        info += " >= ";
        info += std::to_string(bestWeight);
        info += "\nТекущее решение + нижняя оценка МОД на оставшихся вершинах ";
        info += "больше или равна лучшему решению";
        info += "\nдальше эту ветвь продолжать не имеет смысла, отсекаю ее.";
        infoToLogFile(info);
        #endif
        return; // отсечение по верхней границе
    }
    auto& currentEdge = edges[index];
    int vertexU = currentEdge.vertices.first;
    int vertexV = currentEdge.vertices.second;
    #if SHOW_INFO
    info.clear();
    info += "Рассмотрим ребро ";
    info += verticesToStringEdge(vertexU, vertexV);
    infoToLogFile(info);
    #endif
    // Попытка включить ребро в решение
    if (degree[vertexU] < MAX_VERTEX_DEG && degree[vertexV] < MAX_VERTEX_DEG) {
        DSU dsuCopy = dsu;
        if (dsuCopy.unite(vertexU, vertexV)) {
            degree[vertexU]++;
            degree[vertexV]++;
            currentEdges.push_back(currentEdge);

            #if SHOW_INFO
            info.clear();
            info += "Добавляем ребро ";
            info += verticesToStringEdge(vertexU, vertexV);
            info += " с весом ";
            info += std::to_string(currentEdge.weight);
            info += " в текущее решение.";
            infoToLogFile(info);
            #endif
            findMSTRecursive(
                edges, index + 1,
                degree, dsuCopy,
                currentWeight + currentEdge.weight,
                bestWeight,
                currentEdges, bestEdges,
                verticesCount
            );
            #if SHOW_INFO
            info.clear();
            info += "Удаляем ребро ";
            info += verticesToStringEdge(vertexU, vertexV);
            info += " с весом ";
            info += std::to_string(currentEdge.weight);
            info += " из текущего решения";
            infoToLogFile(info);
            #endif
            currentEdges.pop_back();
            degree[vertexU]--;
            degree[vertexV]--;
        }
        #if SHOW_INFO
        else {
            info.clear();
            info += "Ребро ";
            info += verticesToStringEdge(vertexU, vertexV);
            info += " образует цикл, что недопустимо.";
            info += "\nПереходим к рассмотрению следующего ребра.";
```

```
            infoToLogFile(info);
        }
        #endif
    }
    #if SHOW_INFO
    else {
        info.clear();
        info += "Ребро ";
        info += verticesToStringEdge(vertexU, vertexV);
        info += " нарушает правило ограничения на степень вершины.\n";
        info += "Степень вершины ";
        info += ALPHABET[vertexU];
        info += ": ";
        info += std::to_string(degree[vertexU]);
        info += "\nСтепень вершины ";
        info += ALPHABET[vertexV];
        info += ": ";
        info += std::to_string(degree[vertexV]);
        info += "\nПерехожу к рассмотрению следующего ребра.";
        infoToLogFile(info);
    }
    #endif
    // Ветвь без включения текущего ребра
    findMSTRecursive(
        edges, index + 1,
        degree, dsu,
        currentWeight,
        bestWeight,
        currentEdges, bestEdges,
        verticesCount
    );
    #if SHOW_INFO
    info.clear();
    info += "Рассмотрены все ветви рекурсии для ребра ";
    info += verticesToStringEdge(vertexU, vertexV);
    info += ". Функция нашла МОД с весом: ";
    info += bestWeightString(bestWeight);
    info += "\n";
    infoToLogFile(info);
    #endif
}


/**
 * Выводит в стандартный поток результаты работы алгоритмов:
 * точное решение и (если оно отличается) приближённое решение.
 */
void printResult(
    edgesStack& bestEdges,
    edgesStack& approximationEdges,
    int bestWeight,
    int approximationWeight
) {
    std::string result;
    if (bestWeight == approximationWeight) {
        result += "Приближенное и точное значения равны: ";
    }
    else {
        result += "Точное значение: ";
    }
    result += std::to_string(bestWeight);
    result += '\n';
    for (auto& currentEdge : bestEdges) {
```

```
        int vertexU = currentEdge.vertices.first;
        int vertexV = currentEdge.vertices.second;
        result += ALPHABET[vertexU];
        result += " -- ";
        result += ALPHABET[vertexV];
        result += "\t[";
        result += std::to_string(currentEdge.weight);
        result += "]\n";
    }
    if (bestWeight != approximationWeight) {
        result += "Приближенное значение: ";
        result += std::to_string(approximationWeight);
        result += '\n';
        for (auto& currentEdge : approximationEdges) {
            int vertexU = currentEdge.vertices.first;
            int vertexV = currentEdge.vertices.second;
            result += ALPHABET[vertexU];
            result += " -- ";
            result += ALPHABET[vertexV];
            result += "\t[";
            result += std::to_string(currentEdge.weight);
            result += "]\n";
        }
    }
    std::cout << result;
}


/**
 * Основная функция точного поиска МОД с ограничением степени.
 * Считывает матрицу, строит приближённое решение (Краскал с ограничением),
 * затем запускает метод ветвей и границ.
 * Выводит результат и генерирует DOT-файл.
 */
int findMSTBranchAndBound() {
    matrix M = getMatrixFromStream();
    #if SHOW_INFO
    infoToLogFile(matrixToString(M));
    #endif
    if (!isMatrixSymmetric(M)) {
        #if SHOW_INFO
        infoToLogFile("Матрица не симметрична, прерываю");
        #endif
        return EXIT_FAILURE;
    }
    edgesStack edges = getEdgesStack(M);
    sortEdgesStack(edges);
    int verticesCount = M.size();
    std::vector<int> degree(verticesCount, 0);
    DSU dsu(verticesCount);
    edgesStack currentEdges, bestEdges, approximationEdges;
    int approximation = kruskalLowerBound(
        edges,
        dsu,
        verticesCount,
        true,
        &approximationEdges
    );
    int bestWeight = approximation;
    findMSTRecursive(
        edges, 0,
        degree, dsu,
        0,
```

```
        bestWeight,
        currentEdges,
        bestEdges,
        verticesCount
    );
    if (bestWeight == std::numeric_limits<int>::max()) {
        std::cout << "no path\n";
        return EXIT_FAILURE;
    }
    else {
        if (bestEdges.size() == 0) {
            bestEdges = approximationEdges;
        }
        printResult(bestEdges, approximationEdges, bestWeight, approximation);
        edgesToDot(edges, bestEdges, GRAPH_FILENAME);
    }
    return EXIT_SUCCESS;
}


/**
 * Точка входа.
 * Проверяет корректность макроса MAX_VERTEX_DEG и запускает алгоритм.
 */
int main() {
    if (MAX_VERTEX_DEG < 2) {
        #if SHOW_INFO
        infoToLogFile("MAX_VERTEX_DEG < 2, решение невозможно.");
        #endif
        return EXIT_FAILURE;
    }
    return findMSTBranchAndBound();
}
```

Файл generate_matrix.cpp

```
#include <iostream>

#include <vector>

#include <random>

#include <ctime>

#include <fstream>


#ifndef OUTPUT_FILENAME

    #define OUTPUT_FILENAME "./files/matrices/example.txt"

#endif

#ifndef MAX_NUMBER_IN_MATRIX

    #define MAX_NUMBER_IN_MATRIX 100

#endif

#ifndef MATRIX_SIZE

    #define MATRIX_SIZE 11

#endif


using matrix = std::vector<std::vector<int>>;


int getRandomNumber(int mod) {
```

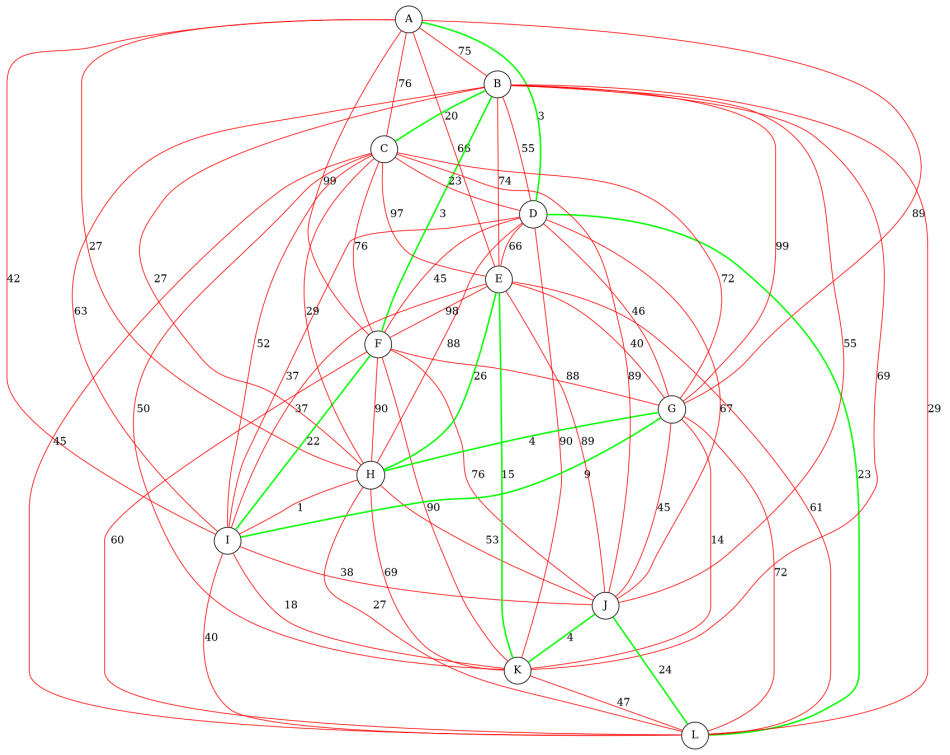
```
    static int counter = 0;
    std::mt19937 engine;
    engine.seed(std::time(nullptr) + ++counter);
    return engine() % mod;
}

void saveMatrix(const matrix& M) {
    std::ofstream out(OUTPUT_FILENAME);
    out << M.size() << '\n';
    for (const auto& row : M) {
        for (int val : row) {
            out << val << ' ';
        }
        out << '\n';
    }
}

int main() {
    // для получения значений от 5 до MATRIX_SIZE + 5 включениено
    int squareMatrixSide = 5 + getRandomNumber(MATRIX_SIZE);
    matrix M(squareMatrixSide, std::vector<int>(squareMatrixSide));
    // цикл по строкам матрицы
    for (int i = 0; i < squareMatrixSide; ++i) {
        // цикл по содержимому строки
        for (int j = i; j < squareMatrixSide; ++j) {
            int randomNumber = 0;
            if (i != j) {
                randomNumber = getRandomNumber(MAX_NUMBER_IN_MATRIX);
            }
            M[i][j] = randomNumber;
            M[j][i] = randomNumber;
        }
    }
    saveMatrix(M);
    return EXIT_SUCCESS;
}
```

ПРИЛОЖЕНИЕ Б

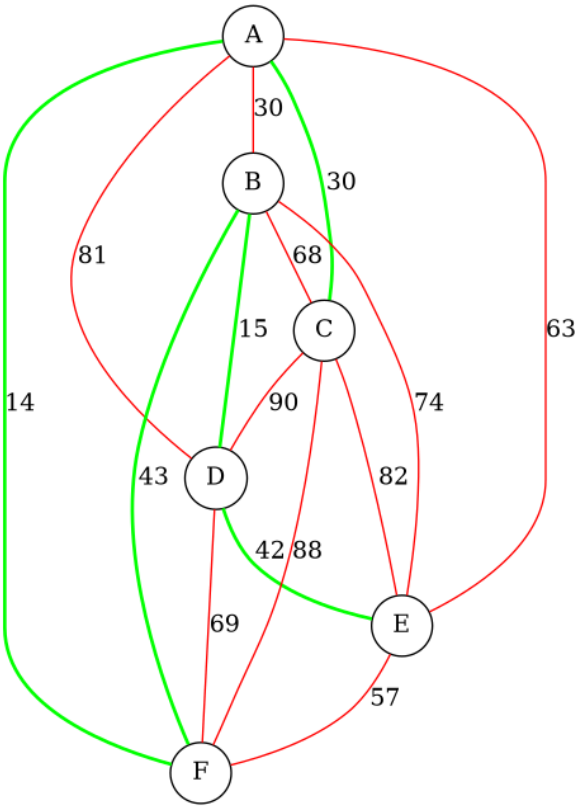
ТЕСТИРОВАНИЕ

Таблица тестирования:

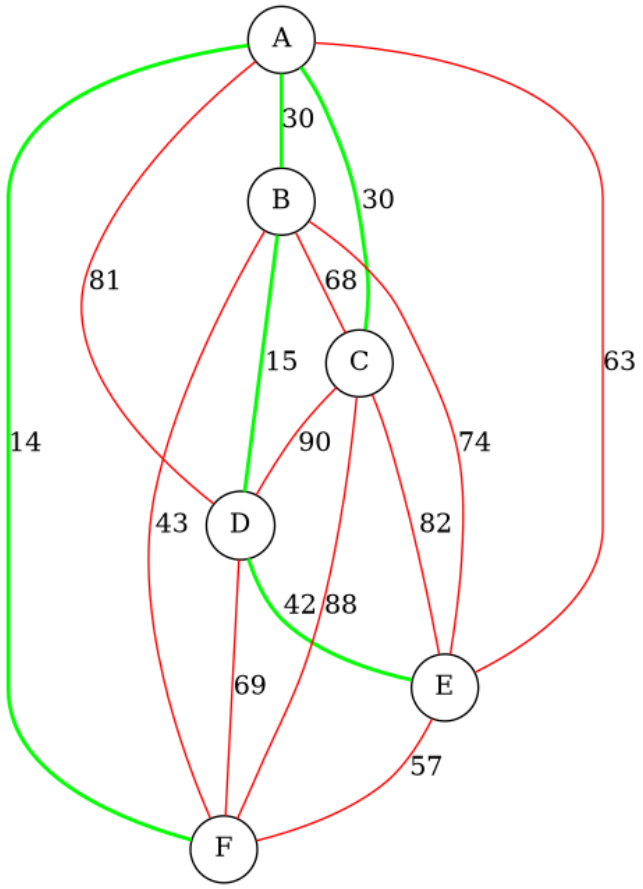
Входные данные:	MAX_VERTEX_DEG = 2 12 0 75 76 3 66 99 89 27 42 0 0 0 75 0 20 55 74 3 99 27 63 55 69 29 76 20 0 23 97 76 72 29 52 89 50 45 3 55 23 0 66 45 46 88 37 67 90 23 66 74 97 66 0 98 40 26 37 89 15 61 99 3 76 45 98 0 88 90 22 76 90 60 89 99 72 46 40 88 0 4 9 45 14 72 27 27 29 88 26 90 4 0 1 53 69 27 42 63 52 37 37 22 9 1 0 38 18 40 0 55 89 67 89 76 45 53 38 0 4 24 0 69 50 90 15 90 14 69 18 4 0 47 0 29 45 23 61 60 72 27 40 24 47 0
Результат:	Точное значение: 153 A -- D [3] B -- F [3] G -- H [4] J -- K [4] G -- I [9] E -- K [15] B -- C [20] F -- I [22] D -- L [23] J -- L [24] E -- H [26] Приближенное значение: 184 H -- I [1] A -- D [3] B -- F [3] G -- H [4] J -- K [4] G -- K [14] B -- C [20] F -- I [22] D -- L [23] J -- L [24] A -- E [66]

Рисунок 1	

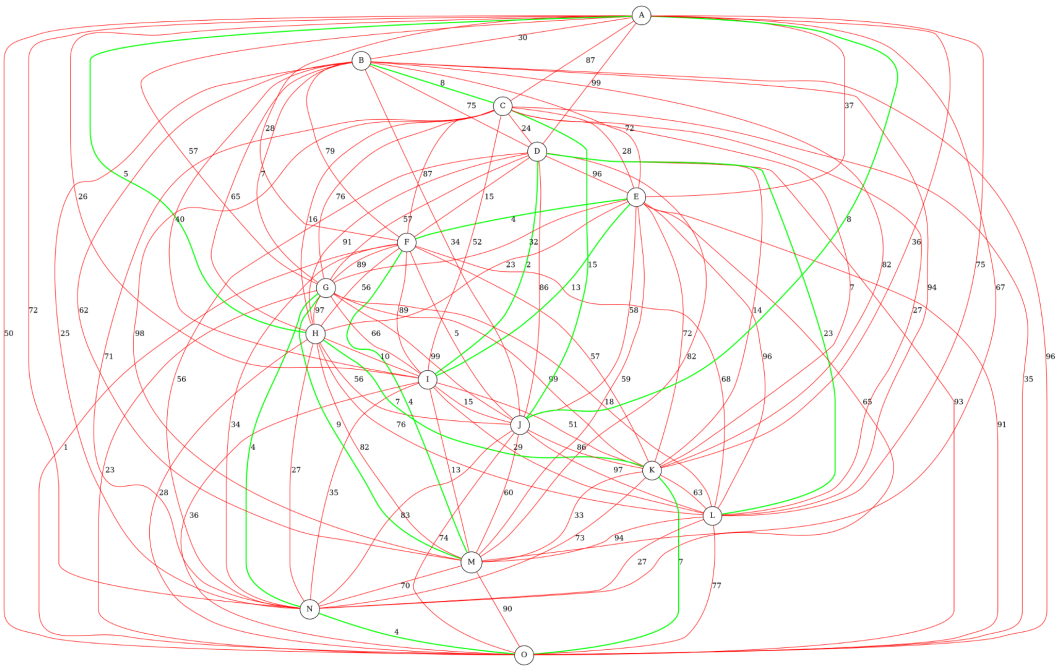
Входные данные:	MAX_VERTEX_DEG = 2 6 0 30 30 81 63 14 30 0 68 15 74 43 30 68 0 90 82 88 81 15 90 0 42 69 63 74 82 42 0 57 14 43 88 69 57 0
Результат:	Точное значение: 144 A -- F [14] B -- D [15] A -- C [30] D -- E [42] B -- F [43] Приближенное значение: 183 A -- F [14] B -- D [15] A -- B [30] D -- E [42] C -- E [82]

Рисунок 2	

Входные данные:	MAX_VERTEX_DEG = 3 6 0 30 30 81 63 14 30 0 68 15 74 43 30 68 0 90 82 88 81 15 90 0 42 69 63 74 82 42 0 57 14 43 88 69 57 0
Результат:	Приближенное и точное значения равны: 131 A -- F [14] B -- D [15] A -- B [30] A -- C [30] D -- E [42]

Рисунок 3	

Входные данные:	MAX_VERTEX_DEG = 2 15 0 30 87 99 37 28 57 5 26 8 36 75 67 72 50 30 0 8 75 72 79 7 65 40 34 82 94 62 25 96 87 8 0 24 28 87 76 16 52 15 7 27 98 71 35 99 75 24 0 96 15 57 91 2 86 14 23 82 56 93 37 72 28 96 0 4 32 23 13 58 72 96 59 65 91 28 79 87 15 4 0 89 56 89 5 57 68 4 34 1 57 7 76 57 32 89 0 97 66 99 99 18 9 4 23 5 65 16 91 23 56 97 0 10 56 7 76 82 27 28 26 40 52 2 13 89 66 10 0 15 51 29 13 35 36 8 34 15 86 58 5 99 56 15 0 86 97 60 83 74 36 82 7 14 72 57 99 7 51 86 0 63 33 73 7 75 94 27 23 96 68 18 76 29 97 63 0 94 27 77 67 62 98 82 59 4 9 82 13 60 33 94 0 70 90 72 25 71 56 65 34 4 27 35 83 73 27 70 0 4 50 96 35 93 91 1 23 28 36 74 7 77 90 4 0
Результат:	Точное значение: 113 D -- I [2] N -- O [4]

	E -- F [4] F -- M [4] G -- N [4] A -- H [5] H -- K [7] K -- O [7] A -- J [8] B -- C [8] G -- M [9] E -- I [13] C -- J [15] D -- L [23] Приближенное значение: 152 F -- O [1] D -- I [2] N -- O [4] E -- F [4] G -- N [4] A -- H [5] C -- K [7] H -- K [7] B -- G [7] A -- J [8] B -- C [8] I -- M [13] D -- L [23] E -- M [59]
Рисунок 3	

Входные данные:	MAX_VERTEX_DEG = 5 15 0 30 87 99 37 28 57 5 26 8 36 75 67 72 50 30 0 8 75 72 79 7 65 40 34 82 94 62 25 96 87 8 0 24 28 87 76 16 52 15 7 27 98 71 35 99 75 24 0 96 15 57 91 2 86 14 23 82 56 93 37 72 28 96 0 4 32 23 13 58 72 96 59 65 91 28 79 87 15 4 0 89 56 89 5 57 68 4 34 1 57 7 76 57 32 89 0 97 66 99 99 18 9 4 23 5 65 16 91 23 56 97 0 10 56 7 76 82 27 28 26 40 52 2 13 89 66 10 0 15 51 29 13 35 36 8 34 15 86 58 5 99 56 15 0 86 97 60 83 74 36 82 7 14 72 57 99 7 51 86 0 63 33 73 7 75 94 27 23 96 68 18 76 29 97 63 0 94 27 77 67 62 98 82 59 4 9 82 13 60 33 94 0 70 90 72 25 71 56 65 34 4 27 35 83 73 27 70 0 4 50 96 35 93 91 1 23 28 36 74 7 77 90 4 0
Результат:	Приближенное и точное значения равны: 85 F -- O [1] D -- I [2] N -- O [4] E -- F [4] F -- M [4] G -- N [4] A -- H [5] F -- J [5] C -- K [7] H -- K [7] K -- O [7] B -- G [7] H -- I [10] G -- L [18]

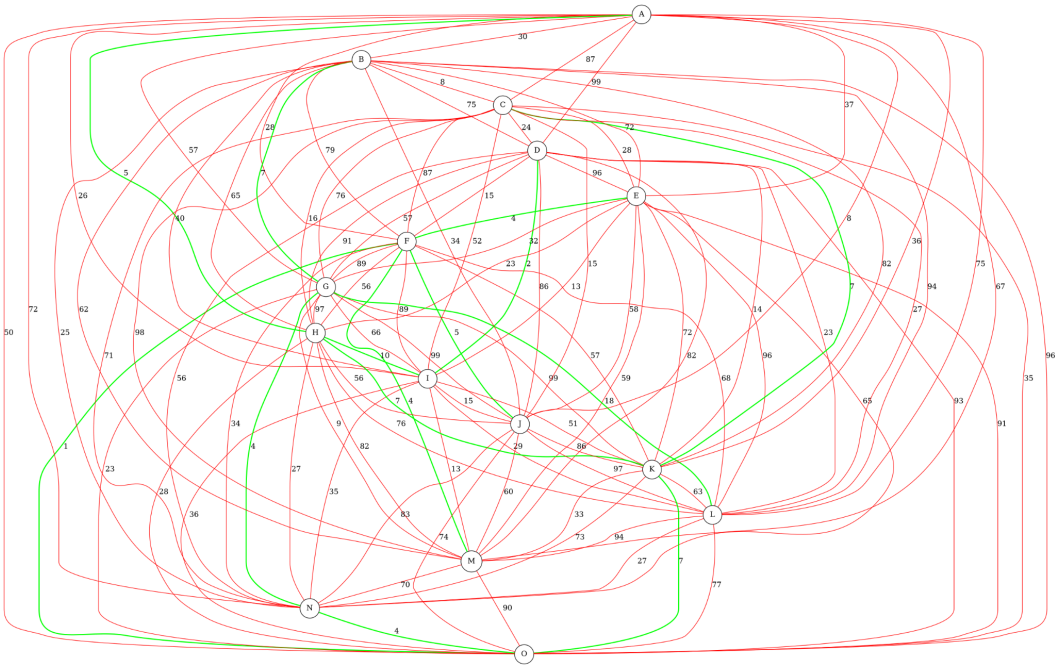



Рисунок 3